PRODUCTION SYSTEMS BLUEPRINT • KAROL JASZTAL · HEAD OF OPERATIONS & AI SYSTEMS LEADER

Inside a Production Multi-Agent System: How I Built an Autonomous Subagent System to Cut Token Burn by 76.8%

Most people build AI setups by dumping 50 pages of instructions, 20 tools, and a bunch of source code into one giant prompt. It burns a ton of cash, runs painfully slow, and makes dumb mistakes. Here is how I do it instead: an autonomous Conductor that hands tiny tasks to fresh subagents with stripped tools, backed by real OS firewalls that kill runaway commands before a terminal ever opens.

VERIFIED DEVELOPER TOKEN CUT

76.8% Less Token Burn

Ditching giant system prompts for tiny frontmatter tags (<1,200 tokens) and loading skills only when needed dropped 50-turn developer sessions from 6.85M tokens down to 1.59M tokens.

23.2% Fresh Subagent Work (1.59M Tokens) **76.8% Pure Waste Cut (5.26M Tokens Saved)**

WORKING TURN CONTEXT SIZE

4,200 Tokens

Keeps memory flat at ~4.2k tokens a turn instead of choking on 82,400+ tokens (−94.9% smaller), so the model never forgets the rules in the middle of a chat.

AVERAGE WAIT TIME PER TURN

4.8s vs 24.2s

−80.2% Faster Replies

Tiny prompt payloads cut developer wait times from 24+ annoying seconds down to under 5 seconds per turn.

RUNAWAY COMMANDS ESCAPED

0 Escapes

100% PreToolUse Hard Blocks

Out-of-band PowerShell hooks instantly block dangerous stuff like `rm -rf`, `git push -f`, and `DROP TABLE` before anything runs.

SPECIALIZED SUBAGENTS

6 Sandboxes

Strict Job Separation

Conductor, Code Builder, Code Reviewer, Security Reviewer, Test Writer, and Researcher with physically stripped tools.

---

01 // ARCHITECTURE & DECOMPOSITION

Ditching the "God Prompt": Why Monolithic Prompts Break Down

Most enterprise AI workflows try to build a single "God Prompt". They paste a 50-page operating manual, 20 API tool schemas, and thousands of lines of code into one massive prompt, and pray the AI doesn't break things. In real life, that fails hard: the model gets confused and forgets rules, costs explode to $34+ a session, and asking it nicely not to delete files is wishful thinking. Here is how my Conductor-Subagent mesh fixes it.

THE MONOLITHIC "GOD PROMPT" TRAP

Single Massive Context (80k–120k Tokens Re-Sent Every Turn)

```
One Generalist Agent • 300 KB Instructions Loaded on Startup
```

How Most People Do It: Force one generalist AI to juggle product specs, coding standards, database schemas, and terminal execution all at once. Every follow-up query retransmits 80,000+ tokens of dead weight, driving costs to $34+ per session and triggering severe hallucination loops.

- **Attention Loss:** The AI forgets rules in the middle of long prompts and starts hallucinating.
- **Burning Cash:** 50 turns burn 6.85M tokens, costing $34.25+ with painful 24-second waits.
- **Fake Safety:** Begging an agent "*please don't break production*" fails when errors happen.

KAROL'S PRODUCTION STANDARD

Autonomous Conductor + Ephemeral Single-Job Subagents

```
1 Task = 1 Clean Window • Physically Stripped Tools • Real OS Hooks
```

How Karol Builds It: An autonomous Conductor breaks work into bite-sized tasks, spawns single-purpose subagents with physically stripped tool manifests, and requires consecutive clean review passes before anything gets merged.

- **Fresh Contexts:** Subagents spawn with 0 prior turns, touch only briefed files, and vanish when done.
- **76.8% Token Cut:** Memory stays flat at ~4.2k tokens, dropping session costs to $7.95 and latency to 4.8s.
- **Unbreakable OS Gates:** Out-of-band PowerShell hooks kill bad commands before a shell ever spawns.

The 4-Stage Lifecycle: How the Conductor Ships Code

STAGE 01

Task Pickup & Scoping

Conductor pulls the next task from Questimus REST API, checks dependencies, and drafts a scoped brief.

STAGE 02

Fresh Builder Spawn

Code Builder spawns in a fresh 0-turn window, writes a failing test first, writes clean code, and proves green.

STAGE 03

Consecutive Clean Pass Review

Two consecutive review rounds must pass 100% clean with zero defects. Whether it takes 2 rounds or 10, no code merges until back-to-back reviews come back completely green.

STAGE 04

Deterministic Push Gate

PreToolUse hooks verify linters and branch rules before atomic git commit and closing the ticket.

Running Across 3 Different Tools: Antigravity, Claude Code, and DSH

HARNESS ENVIRONMENT	FRONTIER REASONING MODELS TESTED & DEPLOYED	PRIMARY OPERATIONAL ROLE	HOOK ENFORCEMENT MECHANISM
Google Antigravity	`Gemini 4 Argon, Gemini 3.8 Flash / Pro, Google AI Studio reasoning engines, plus earlier generational baselines (Gemini 1.5 Pro, and several older ones)`	Visual application builds, multi-workspace refactoring, and interactive agent collaboration	`PreToolUse & PostToolUse JSON IPC via .agents/hooks.json`

HARNESS ENVIRONMENT	FRONTIER REASONING MODELS TESTED & DEPLOYED	PRIMARY OPERATIONAL ROLE	HOOK ENFORCEMENT MECHANISM
Claude Code (Anthropic)	`Sonnet 5, Sonnet 5.5, Opus 4.8, Opus 5, Opus 5.5, Fable 5, plus earlier frontier baselines (Claude 3 Opus, and several older ones)`	Deep logic reviews, complex test suites, and autonomous conductor runs	`PowerShell scripts registered in .claude/settings.json`
DeepSeek Harness (DSH)	`DeepSeek v4 Pro, DeepSeek v4 Flash, DeepSeek v3, GLM 5.3, GLM 5.3 Flash, local Ollama/Unsloth setups, plus earlier open-weight baselines (and several older ones)`	Fast background checks, local test runs, and rapid code reviews	`Headless CLI wrapper with strict standard output validation`

02 // LEAST PRIVILEGE & SANDBOXING

Physical Tool Stripping: Why Telling Agents "Don't Edit Files" Fails

Beginner agent setups give every model full system access (Read, Write, Bash, Git, Network) and then try to tell it in plain English: "*Please only review code, do not edit files.*" That is a complete joke. When an agent hallucinates or encounters an edge case, it will edit files anyway. In my setup, we use physical tool stripping: if an auditor agent does not need to edit files, the `write_file` tool is physically deleted from its manifest. Even if an adversarial prompt tries to hijack it, the tool literally does not exist.

SUBAGENT ROLE	TARGET MODELS	SANDBOXED TOOLS ALLOWED	PHYSICALLY STRIPPED TOOLS	LIFECYCLE MODEL
Conductor	`Gemini Flash / Sonnet / Opus`	`Task planning, Questimus REST, subagent invoke`	`Direct code writes, raw git commits`	Persistent (issue supervisor)
Code Builder	`Model-inherited (Sonnet/Flash)`	`Scoped Read, Scoped Write, Test Runner`	`Git push, branch switches, external HTTP`	Single-Task (1 task = 1 fresh turn)
Code Reviewer	`Model-inherited (Opus/Sonnet)`	`File Read, Git Diff, Grep/Glob search`	`File Write, File Edit, Shell exec, Git push`	Throwaway (Read-only review fork)
Security Reviewer	`Frontier reasoning engine`	`AST Inspector, File Read, Secrets Scanner`	`File Write, File Edit, Shell, Network egress`	Throwaway (Read-only pre-commit)
Test Writer	`High-speed reasoning model`	`File Read, Scoped Test Dir File Write`	`Production source edits, Git operations`	Single-Task (Sandboxed generator)

SUBAGENT ROLE	TARGET MODELS	SANDBOXED TOOLS ALLOWED	PHYSICALLY STRIPPED TOOLS	LIFECYCLE MODEL
Research Subagent	`High-efficiency model`	`Web Search, Documentation Read, Grep`	`File Write, Shell exec, Git operations`	Background (Dies when search finishes)

Production Operational Contracts: Exact Deployed Directives

LEAST PRIVILEGE ENFORCEMENT & AUTONOMOUS SPECIALIZATION

Production Agent Directives: Maximum Autonomous Power vs. Purposeful Limitations

Real production specifications from `AGENT.md` and atomic specialist files. Notice the intentional architecture: high-leverage autonomous power (parallel specialist swarms, red-before-green TDD, Questimus REST state) paired with unbreakable boundaries (physically stripped write tools, zero git write access, strict scope locks).

01 Code Reviewer Contract: 12-Specialist Swarm Dispatcher with Stripped Write Tools

`.agents/agents/code-reviewer/AGENT.md` (220-Line Production Contract) • Coordinates 12 atomic domain specialist subagents in parallel.

⚡ HOW POWERED IT IS (AUTONOMOUS ENGINE)

- **Parallel Specialist Swarms:** Dispatches up to 12 domain subagents in one turn with strict JSON output schemas.
- **Round-over-Round Diffing:** Tracks ✅ `Confirmed Fixed`, 🟡 `Partially Fixed`, and ⚠️ `New Regression` across rounds.
- **Systemic Pattern Sweeps:** Auto-greps whole repo for defect classes the second a regex or $O(n^2)$ bug is found.

🔴 WHERE IT IS PURPOSEFULLY LIMITED (HARD GUARDRAILS)

- **Physical Tool Stripping:** `write_to_file` and `replace_file_content` are literally deleted from allowed tools.
- **Read-Only Shell:** Destructive commands blocked; shell access is restricted to read-only audits.
- **Zero Delegation Leakage:** The `agent` grant is locked exclusively to specialist dispatch—cannot delegate its own review.

</> .agents/agents/code-reviewer/AGENT.md (Production Directives Excerpt)

```
---
name: code-reviewer
description: Independent code review subagent — correctness, edge cases, security,
performance.
model: inherit
tools:
  - view_file
  - run_command
  - invoke_subagent
  # NOTICE: write_to_file and replace_file_content are PHYSICALLY STRIPPED from tools
---

## Cold Review Mandate & Zero Rationalization
You are a staff-level code reviewer doing a pre-delivery review. You have no knowledge
of what the implementing session intended or rationalized. You review the code as it
is,
```

```
cold. Never soften findings because the code "seems to work" or "is probably fine."

## Specialist Dispatch (On by Default — Parallel Subagent Swarm)
Beyond your own 7 Review Dimensions, decide which bundled specialist checklists in
<AGENT_DIR>/resources/specialists/ are relevant and dispatch EACH as a parallel
subagent:
- security.md        — auth/session/token/role changes or backend diff > 100 lines
- performance.md     — algorithmic complexity, hot-path allocations, unindexed filters
- red-team.md        — adversarial attack vectors, silent failures, trust boundary
exploits
- error-handling.md  — exception swallowing, unhandled promise rejections, recovery
chains
- api-contract.md    — API/OpenAPI/GraphQL breaking changes and backward compatibility
- data-migration.md  — schema/migration changes, zero-downtime rollbacks

Dispatch relevant specialists in a SINGLE message so they execute in parallel.
Each specialist subagent returns strict JSON:
{"severity":"CRITICAL|INFORMATIONAL","confidence":N,"path":"...","line":N,...}
Merge specialist findings into your own 3-tier output; dedupe across all sources.

## Systemic Pattern Sweep (Mandatory on Defect Discovery)
The moment you flag a regex or algorithmic defect (e.g. an O(n²) pattern or bad
complexity),
immediately grep the REST of the codebase for the same pattern before finishing the
round.
A fix in one file never implies the pattern was swept everywhere else it appears.

## Follow-Up Round Comparison Engine
When a prior report is supplied, track state transitions explicitly:
- ✅ CONFIRMED FIXED — what was wrong, confirmed resolved with evidence
- 🟡 PARTIALLY FIXED  — what changed, what's still incomplete or wrong
- 🆕 NEW INDEPENDENT — issue surfaced by this round's broadened scope
- ⚠️ NEW REGRESSION  — defect directly introduced by a prior round's fix
```

---

02 Code Builder Implementation Contract: Test-First Engine with Absolute Git Lockdown

`.agents/agents/code-builder/AGENT.md` (132-Line Production Contract) • Conductor Task Executor

⚡ HOW POWERED IT IS (AUTONOMOUS ENGINE)

- **Live Questimus API Ingestion:** First step is fetching the authoritative task directly via node `questimus.js get <id>`.
- **Mandatory Red-Before-Green TDD:** Must write a failing test first, run it, and paste real terminal output before implementing.
- **Strict DoD Gate:** Non-negotiable typecheck, lint, and test pass; failure of any linter is an immediate task failure.

🛑 WHERE IT IS PURPOSEFULLY LIMITED (HARD GUARDRAILS)

- **Absolute Git Lockdown:** Explicitly forbidden: `NEVER git add`, `NEVER git commit`, `NEVER push`. Changes left unstaged.
- **Scope Lock:** Builds ONLY briefed files. If task exceeds file boundaries, it must STOP and report—cannot touch outside files.
- **Ephemeral 1-Task Lifetime:** 0-turn fresh context window per task; completely dies upon outputting DEBUG REPORT.

</> .agents/agents/code-builder/AGENT.md (Production Directives Excerpt)

```
---
name: code-builder
description: Implementation subagent for conductor workflows — one task per spawn,
always a fresh context.
model: inherit
tools:
  - view_file
  - replace_file_content
  - write_to_file
  - run_command
---

## Authority & First Step: Direct Questimus REST Ingestion
The brief's task block does not carry the task's full text: it carries the Questimus
sub-issue
identifier. Your FIRST STEP before any other work: read that exact sub-issue via the
questimus skill:
  node <path-to-questimus.js> get <id>
What you read directly from Questimus is authoritative and complete.

## Absolute Branch & Git Lockdown
Work on the branch the brief states — never create, switch, or delete branches.
YOU NEVER git add, NEVER git commit, NEVER push.
The conductor reviews the working tree, stages, and commits. Leave every change
unstaged.

## Scope Lock & Environment Freeze
Build ONLY the task's declared Files. If you find the task genuinely cannot be done
within that list, STOP and report that fact — do not expand scope to make it work.
Fixes stay minimal and inside the Files list; unrelated bugs are reported, never
fixed.
The conductor's environment state is binding: if impossible, STOP and report.

## Mandatory Red-Before-Green TDD
When the task prescribes TDD: write the failing test first and show it RED before
implementing.
The red run's output MUST be pasted in your Evidence — a red-before-green claim
without
the pasted red output is UNPROVEN and rejected.

## Non-Negotiable Output: DEBUG REPORT Block
End every single run with exactly this structured block:
  ## DEBUG REPORT — <task id>
  **What was built**: <summary>
  **Files touched**: <exact list — must match task's declared Files>
  **Evidence**:
```

```
- <command> → <pasted real output, truncated only for length, never summarized>
**Status**: verified | best-effort | reverted
```

---

03 Atomic Specialist Swarm: 12 Domain Checklists with Strict JSON Schemas

`.agents/agents/code-reviewer/resources/specialists/*.md` • Decoupled, single-responsibility review protocols executed in parallel.

HOW POWERED IT IS (12 DEEP DOMAIN ENGINES)

- **12 Atomic Domain Checklists:** `security.md`, `red-team.md`, `performance.md`, `api-contract.md`, `data-migration.md`, `error-handling.md`, etc.
- **Adversarial Red Teaming:** Probes happy-path stress, silent failure swallows, and trust boundary exploits.
- **Machine-Parsable JSON:** Specialists return pure JSON lines with confidence scores (1-10) for automated deduplication.

WHERE IT IS PURPOSEFULLY LIMITED (HARD GUARDRAILS)

- **Strict Domain Scope:** Specialists cannot wander into other areas; they execute only their declared checklist.
- **Zero Environment Mutation:** Read-only access to supplied code snippets; zero shell execution or filesystem access.
- **Zero Chatter:** Output is restricted to valid JSON findings or `NO FINDINGS`.

</> resources/specialists/security.md & red-team.md (Production Directives Excerpt)

```
# Output Contract Across All 12 Specialists (Strict JSON Lines):
{"severity":"CRITICAL|INFORMATIONAL","confidence":N,"path":"file","line":N,"category":"se
If no findings: output `NO FINDINGS` and nothing else. No preamble, no commentary.

---
### Excerpt: security.md (Input Validation & Trust Boundaries)
Scope: When SCOPE_AUTH=true OR (SCOPE_BACKEND=true AND diff > 100 lines)
- Input Validation: query parameters used directly in file paths; unvalidated webhook
payloads
- Auth & Authorization: endpoints missing auth middleware; checks defaulting to allow
- Role Escalation: user can modify their own role/permissions or access IDOR objects
- Injection Vectors: command injection via subprocess calls; SSRF via user-controlled
fetch
- Cryptographic Misuse: weak hashing, non-constant-time comparisons on secrets/tokens
- Secrets Exposure: credentials in URLs; PII stored in plaintext

---
### Excerpt: red-team.md (Adversarial Stress Testing)
This is NOT a checklist review. This is adversarial analysis.
You have access to the other specialists' findings. Your job is to find what they
MISSED.
Think like an attacker, a chaos engineer, and a hostile QA tester simultaneously:
1. Attack the Happy Path: What happens under 10x normal load? Two requests hit same
resource?
2. Find Silent Failures: Catch-all blocks that swallow exceptions? Partial batch
crashes?
3. Exploit Trust Assumptions: Data validated on frontend but trusted blindly on
```

```
backend?
4. Find What Specialists Missed: Cross-category seams where two separate systems meet.
```

03 // OPERATING SYSTEM DEFENSE

Non-Bypassable OS Hooks: Stopping Disasters Before the Shell Spawns

Natural-language guardrails (e.g. "*You are a helpful assistant. Do not execute destructive commands*") are wishful thinking. In complex codebases, recursive error loops, adversarial prompt injection, and hallucinations will eventually bypass text rules. Our rule is dead simple: **Never trust an AI to self-police. Deterministic code must govern probabilistic models—put a real script in front of the model that slams the door shut.**

PreToolUse Interception Architecture (JSON IPC Hook)

STAGE 01

Tool Call Request

Autonomous agent requests shell execution (e.g., `run_command("rm -rf ./build")`).

STAGE 02

Payload Interception

Harness intercepts execution request out-of-band and pipes JSON payload into STDIN of `security-gate.ps1`.

STAGE 03

Deterministic Regex Audit

Hook parses command string against 19 destructive regex signatures (force-push, disk wipes, hard resets).

STAGE 04

Exit Code 2 Hard Block

Destructive pattern detected: hook emits exit code 2 to stderr. The OS shell never spawns. Zero damage possible.

Active Blocked Signatures in Production (Exit Code 2 Hard Blocks)

THREAT CATEGORY	TARGET SIGNATURE (REGEX PATTERN)	OPERATIONAL RISK MITIGATED	HOOK ACTION
Filesystem Wipe	`rm -rf, rm -fr, sudo rm`	Irreversible deletion of build assets or project source trees	EXIT CODE 2
Git History Destruction	`git push --force, git push -f, git reset --hard`	Overwriting remote commit history or uncommitted working state	EXIT CODE 2

THREAT CATEGORY	TARGET SIGNATURE (REGEX PATTERN)	OPERATIONAL RISK MITIGATED	HOOK ACTION
Branch Corruption	`git branch -D`	Force-deleting branches containing unmerged production code	EXIT CODE 2
Working Tree Wipe	`git checkout \., git restore \., git clean -f`	Silently discarding uncommitted local engineering work	EXIT CODE 2
Database Annihilation	`DROP TABLE, DROP DATABASE, TRUNCATE TABLE`	Catastrophic drop of relational tables in local or container databases	EXIT CODE 2
Host System Damage & Fork Bombs	`mkfs, dd if=, chmod -R 777, :\(\)\{:|:&\};:`	Filesystem corruption, permission stripping, and fork bombs	EXIT CODE 2

Production PreToolUse Hook Implementation (Verified Real PowerShell Script)

</> .agents/hooks/security-gate/security-gate.ps1

```
# security-gate.ps1 — PreToolUse hook, blocks destructive Bash / run_command commands
# Hook event: PreToolUse → bash | run_command
# Exit 2 = hard block (Claude Code/DSH); Exit 0 with decision: "deny" (Antigravity)

$ErrorActionPreference = 'Stop'
$InputJson = [Console]::In.ReadToEnd()

$cmd = ''
$isAntigravity = $false
try {
    $data = $InputJson | ConvertFrom-Json
    if ($null -ne $data.tool_input -and $null -ne $data.tool_input.command) {
        $cmd = [string]$data.tool_input.command
    } elseif ($null -ne $data.toolCall -and $null -ne $data.toolCall.args -and $null -ne
$data.toolCall.args.CommandLine) {
        $cmd = [string]$data.toolCall.args.CommandLine
        $isAntigravity = $true
    }
} catch {
    $cmd = ''
}

# 19 Patterns that are hard-blocked regardless of context (case-insensitive)
$patterns = @(
    'rm -rf', 'rm -fr', 'git push --force', 'git push -f ', 'git push -f$',
    'git reset --hard', 'git clean -fd', 'git clean -f', 'git branch -D',
    'git checkout \.', 'git restore \.', 'DROP TABLE', 'DROP DATABASE',
    'TRUNCATE TABLE', 'DELETE FROM .* WHERE 1=1', 'mkfs', 'dd if=',
    'chmod -R 777', 'sudo rm', ':\(\)\{:\|:&\};:'
)
```

```
foreach ($pat in $patterns) {
    if ($cmd -match "(?i)$pat") {
        $msg = "BLOCKED by security-gate: destructive pattern detected ($pat)."
        [Console]::Error.WriteLine($msg)
        [Console]::Error.WriteLine("   Pattern: $pat")
        [Console]::Error.WriteLine("   Command: $cmd")
        [Console]::Error.WriteLine("   If this is intentional, run it manually in your
terminal.")

        if (-not $isAntigravity) {
            exit 2 # Claude Code & DSH: Exit 2 halts execution immediately
        }
        @{ decision = "deny"; reason = "$msg Command: $cmd" } | ConvertTo-Json -Compress
        exit 0
    }
}

if ($isAntigravity) { @{ decision = "allow" } | ConvertTo-Json -Compress }
exit 0 # Verified clean: tool allowed
```

04 // FINOPS & CONTEXT MODELING

Slashing 76.8% of API Waste: Dynamic Injection vs. Prompt Stuffing

I have run over 1,000 autonomous agent sessions to date. The first few hundred were messy baby steps—lots of trial and error, breaking builds, testing crazy setups, and finding shit out the hard way. But the last 700+ sessions have run exclusively on this mature multi-agent architecture. Our full engineering library contains 16 specialized skills totaling 312 KB of instructions (~78,000 tokens). Statically stuffing all of that into every turn burns through your budget in minutes. Here is the real data from 700+ production sessions comparing the old monolithic approach to our dynamic injection engine.

TOTAL INSTRUCTION SIZE

Full Skill Library Footprint

312.4 KB

~78,000 tokens across 16 specialized manuals (code review, database migrations, security audits, git release).

MONOLITHIC PROMPT STUFFING

Static God-Prompt Baseline

82,400 Tokens/Turn

6.85 million tokens per 50-turn session ($34.25 API burn). Sluggish 24.2-second wait times per response.

DYNAMIC SKILL INJECTION

Karol's Dynamic Engine

4,200 Tokens/Turn

1.59 million tokens per 50-turn session ($7.95 API cost). **−76.8% cost cut** and instant 4.8s response times.

Empirical Benchmark Study: 700+ Production Engineering Sessions

OPERATIONAL METRIC	MONOLITHIC BASELINE	DYNAMIC SUBAGENT MESH	DELTA	IMPACT
Baseline Turn Context Size	82,400 tokens	4,200 tokens	-78,200 tokens	-94.9%
Total Tokens per 50-Turn Session	6,850,000 tokens	1,590,000 tokens	-5,260,000 tokens	-76.8%
Average Turn Response Latency	24.2 seconds	4.8 seconds	-19.4 seconds	-80.2%
Attention Retrieval Accuracy (Pass@1)	61.4%	94.2%	+32.8%	+53.4%
False-Positive Tool Invocations	14.2%	0.4%	-13.8%	-97.2%
AVERAGE DEVELOPER SESSION COST	$34.25	$7.95	-$26.30	-76.8%

05 // STATE PERSISTENCE & RESUMPTION

Stateless LLMs, Persistent Memory: Wiping Context to 0 Tokens Without Losing Progress

Large language models are stateless compute engines, not databases. If you keep 50 turns of back-and-forth chat in memory, the context window fills up with junk and the model starts drifting. In my architecture, we can wipe the agent's context completely to **0 tokens** at any time without losing a single line of progress. All operational state is externalized to **Questimus (our internal project management, ticketing and tracking tool)** via REST API through a clean two-level continuity model.

LEVEL 01: MACRO STATE

The Project HANDOFF Page

Stores high-level system state, current release version, active git branch, verified test suites, and discovered landmines. Checked automatically at session start; serialized and committed via REST at session close.

```
node questimus.js handoff •
Restores full session in <200ms
```

LEVEL 02: MICRO STATE

Atomic Ticket Description Appends

Mid-task progress, failing test output, and fix evidence are appended directly into the active ticket description (safe description-only editing; never uneditable comments). Any fresh agent can resume mid-task without reading conversation logs.

```
node questimus.js update <id> --
desc-file .tmp/progress.md
```

Command-Line Interface State Protocol (Questimus REST Wrapper)

</> State Serialization Commands (PowerShell)

```
# 1. Read macro session context at startup (restores state in <200ms)
node .agents/skills/questimus-arsenal/scripts/questimus.js handoff

# 2. Append in-flight atomic progress to active ticket description
node .agents/skills/questimus-arsenal/scripts/questimus.js update TASK-104 --desc-file
.tmp/progress.md

# 3. Serialize end-of-session run state to persistent database
node .agents/skills/questimus-arsenal/scripts/questimus.js handoff --write-file .tmp/handoff-
state.md
```

06 // VIDEO WALKTHROUGH & LIVE PROOF

Live Screencast Walkthrough: Multi-Agent Systems in Action

A high-signal, 7-minute split-screen walkthrough demonstrating autonomous subagent delegation, live PreToolUse security gate interception, and dynamic skill injection against production codebases.

COMING SOON



7-Minute Multi-Agent Systems Architecture Screencast

Featuring split-screen terminal execution, real-time `PreToolUse` security intercepts, live context token profiling, and zero-context session recovery.

`Presenter: Karol Jasztal • Target Duration: 6:45`

Director's Script & 5-Beat Timestamp Index

TIMESTAMP	CHAPTER TITLE	CORE DEMONSTRATION & KEY INSIGHT
`0:00 - 1:15`	**The Monolithic Prompt Trap**	Why 90% of enterprise agent setups stall: attention degradation ("Lost in the Middle") and runaway API costs.
`1:15 - 3:15`	**Subagents & The Conductor Pattern**	Live demonstration of physical tool stripping: auditor agents operate with zero write or shell tools.
`3:15 - 4:45`	**Deterministic OS Guardrails in Action**	Simulating an agent requesting `rm -rf`; out-of-band PowerShell hook traps command at Exit Code 2.
`4:45 - 6:15`	**Token Economics & Session Continuity**	Dynamic skill injection slashing 76.8% of token burn, paired with 0-token context wipes via Questimus REST.
`6:15 - 7:00`	**Strategic Operations Conclusion**	Executive hiring takeaway: moving from fragile prompt experimentation to industrial software systems.

Pre-Flight Live Demonstration Commands (Verified Real Output)

DEMO 01

Tool Boundary Audit

Verifies `code-reviewer` manifest. Confirms `write_to_file` and `replace_file_content` are physically stripped.

```
Get-Content .agents/agents/code-reviewer/AGENT.md -Head 25
```

DEMO 02

Security Gate Intercept

Simulates agent attempting `rm -rf ./production_build`. Intercepted out-of-band: shell never spawns, Exit Code 2.

```
>> HARD-BLOCKED (Exit Code 2) • Shell NEVER spawned <<
```

DEMO 03

Dynamic Registry Proof

Measures 16 production skills: 312.4 KB (~78,000 tokens) decoupled from static prompts into lightweight on-demand triggers.

```
16 Skills | 312.4 KB decoupled | <1.2k tokens on startup
```

Executive Hiring Dossier: Three Proven Operational Capabilities

01 // SYSTEMS LEADERSHIP

Architecting Autonomous Workflows

Breaking huge engineering roadmaps into bite-sized tasks that fresh subagents build and verify with back-to-back clean reviews.

02 // FINOPS CONTROL

Mathematical Token Modeling

Cutting API bills by 76.8% by feeding models only what they need, exactly when they need it, with zero memory bloat.

03 // DEFENSE-IN-DEPTH

Non-Bypassable OS Security

Stripping unneeded tools from subagents and running real OS hooks so an AI can never break production or wipe a drive.

© 2026 Karol Jasztal • Hosted on GitHub Pages • All Rights Reserved